

Python Programming Assignment - 1

1. Who developed Python and why was it created? Explain one major difference between Python and C in terms of program execution.

Ans:

Python was developed by Guido van Rossum. It was created to address issues found in the ABC language and to make programming easier.

Difference between Python and C:

- Python is an interpreted language and executes code line-by-line without prior compilation.
- C is a compiled language.

2. Explain the Python build process starting from the .py file to execution, mentioning the role of:

- Python Interpreter
- Bytecode
- PVM (Python Virtual Machine)

Ans:

When a developer writes code in a .py file:

- The **Python Interpreter** performs lexical analysis - it scans the code and breaks it down line by line.
- The resulting tokens are parsed and transformed into an Abstract Syntax Tree (AST). Syntax errors are checked during this parsing stage.
- The AST then goes through **bytecode** compilation. This creates .pyc files, which are stored inside the `__pycache__` folder.
- The **PVM (Python Virtual Machine)** reads the bytecode and executes it line-by-line. It handles runtime actions such as memory management, method calls, and control flow.

Build Process - Flow Diagram:

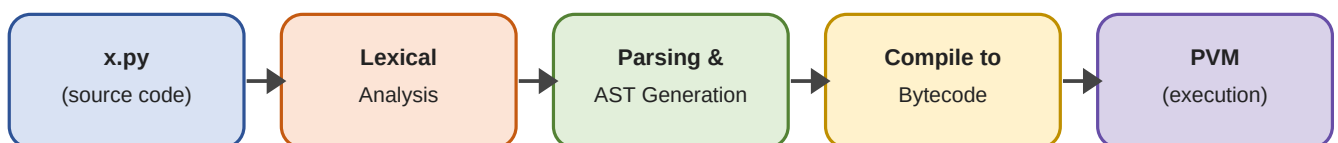


Figure: Flow of a Python program from source code (.py) through lexical analysis, parsing/AST generation, and bytecode compilation, to final execution by the PVM.

3. What is bytecode in Python? Where is it stored, and why does Python use it?

Ans:

When the developer's source code in a .py file is processed, the Python interpreter scans the code and generates the Abstract Syntax Tree (AST). This AST is then compiled into bytecode (.pyc file).

Bytecode is stored in the `__pycache__` folder. Python uses bytecode because it is a lower-level, platform-independent representation that the PVM can execute faster than re-parsing the raw source code every time.

4. Is Python a compiled language or an interpreted language? Justify your answer with execution steps.

Ans:

Python is an interpreted language.

Step 1 - Developer writes code in a .py file.

Step 2 - The code is converted into smaller tokens.

Step 3 - The AST is generated and validated for syntax errors.

Step 4 - The AST is converted/compiled into bytecode.

Step 5 - The bytecode is stored as a .pyc file under the __pycache__ folder.

Step 6 - The PVM executes the bytecode line-by-line.

5. What is the purpose of the __pycache__ folder?

Ans:

The __pycache__ folder automatically stores the compiled bytecode version of a module as a .pyc file, so Python does not need to recompile the source code every time it is run (if it has not changed).

6. Python is called a dynamically typed language. Explain this with a short code example.

Ans:

```
a = 8          # int
b = "vishal"   # str
c = 8.50       # float
```

In the above examples, it is not required to explicitly declare whether a variable is a string, integer, or decimal value - Python automatically identifies the value and assigns its type at runtime.

7. Explain why Python is considered:

- Platform independent
- Portable

Ans:

Python is platform independent because Python programs can run on Windows, Linux, and macOS without changing the code. This is what makes Python portable across different systems.

8. What do you understand by "indentation-specific language"? Why is indentation mandatory in Python?

Ans:

An indentation-specific language uses whitespace (indentation) to structure the program, instead of symbols like curly braces.

In Python, this removes the need for extra symbols and makes development easier and more readable.

Indentation is mandatory in Python because it defines the structure and scope of the code (for example, which statements belong inside a loop, function, or conditional block).

9. What will happen if indentation is not maintained properly? Explain with a small code snippet.

Ans:

If indentation is not maintained, the program will fail and throw a compile-time error.

Example 1 (correct indentation):

```
a = 30
b = 35
if b > a:
    print("b is greater than a")
```

Example 2 (incorrect indentation):

```
a = 30
b = 35
if b > a:
print("b is greater than a")
```

- In Example 1, indentation is used for the print statement, so the program runs successfully.
- In Example 2, indentation is not used, so the program fails and raises a compile-time error.

Error: IndentationError: expected an indented block after 'if' statement on line 1

10. Python supports multiple programming paradigms. Name any three paradigms and explain one in brief.

Ans:

1. Simple and easy to learn - Python has simple, English-like syntax that is easy for beginners to understand and write programs.
2. High-level programming language - Python hides complex details like memory management, allowing programmers to focus on logic and problem-solving instead of hardware.
3. Interpreted language - Python code is executed line-by-line using an interpreter. This makes debugging easy because errors are shown immediately.